

Supplementary Information

No Small Observer Can Verify a Black Hole Firewall

Michael T. Bennett^{1,*}

¹School of Computing, The Australian National University, Canberra ACT 2601, Australia

*Corresponding author: m@michaeltimothybennett.com

This document contains the supporting formal material for the main text. Section S1 gives the minimal bookkeeping kernel used by the main text. Section S2 proves the capacity complementarity inequality used to interpret complementarity as a join failure. Section S3 formalises the difference between a single patch verifier and a distributed protocol using explicit verification ingredients, occurrence versus co-instantiation, and a bounded-history version of the decoder-key argument. That last step is where the recent *Time Is Space* identities matter most [1, 2]. Section S4 gives additional derivations and robustness checks for the AMPS ledger. Section S5 documents the computations that generate Figure 1 and Extended Data Figures 1 and 2. Section S6 gives a cautious outlook on how covariant capacity bookkeeping can be used as a constraint on quantum gravity descriptions.

Interpretation. If you are coming from outside quantum gravity, start with the Glossary and notation section in the main text. It defines causal patch, decoder, code subspace, covariant entropy bound, and the parameters ρ and α_* . This supplement is more technical, but the core picture is simple. Single-observer verification requires one joint physical record. That is stronger than ingredient-wise occurrence across a history. And if the protocol tries to serialise the decoder choice across a bounded preverification window, *Time Is Space* tells us that the bookkeeping burden has only moved into a history vocabulary, not vanished [2].

S1 Minimal bookkeeping kernel

This section states the exact bookkeeping kernel used by the main text. Rather than restating a parallel notation, I use the same shared kernel file as the manuscript so the supplement and manuscript literally share the same definitions. That removes an avoidable source of drift.

Interpretation. The role of the framework in this paper is narrow. It is the bookkeeping language that lets me talk about finite embodied descriptions, joint realisability, and control-state capacity inside one patch. So the only primitives I include here are the ones the theorem actually uses.

S1.1 Shared kernel

Definition S1.1 (Environment, programs, and vocabularies). An *environment* is a nonempty set Φ of mutually exclusive states. It is assumed infinite by default. A (*declarative*) *program* is any set of states $p \subseteq \Phi$. For any set X , write 2^X for the set of all subsets of X . In particular, let $\mathcal{P} := 2^\Phi$ denote the set of all programs.

A *fact* about a state ϕ is a program that contains ϕ . An *aspect* of the environment is a set x of programs such that

$$x \in \{l \subseteq \mathcal{P} \mid \bigcap_{p \in l} p \neq \emptyset\}.$$

We say an aspect x *exists at* ϕ if every program in x is a fact about ϕ . A *vocabulary* is any set of programs $v \subseteq \mathcal{P}$. Unless specified otherwise, vocabularies and aspects are assumed finite.

Interpretation. Φ is the set of possible situations. A program is one constraint. It is the set of situations where that constraint holds. A vocabulary is the list of constraints you can actually use.

Definition S1.2 (Abstraction layer, statements, and truth sets). A vocabulary v induces an embodied language

$$L_v = \{l \subseteq v \mid \bigcap_{p \in l} p \neq \emptyset\}.$$

Elements $l \in L_v$ are called *statements*. Unless specified otherwise, statements are assumed finite.

Statements behave like conjunctions of vocabulary programs. The *truth set* of l is

$$T(l) := \bigcap_{p \in l} p \subseteq \Phi,$$

so l is *true* at $\phi \in \Phi$ if and only if $\phi \in T(l)$. We use the convention $T(\emptyset) := \Phi$.

Interpretation. A statement is a bundle of vocabulary constraints that can all hold at once. $T(l)$ is the set of situations where the whole bundle holds.

Definition S1.3 (Completions, extensions, and equivalence). A *completion* of a statement $x \in L_v$ is any statement $y \in L_v$ such that $x \subseteq y$. The *extension* of x is the set of its completions

$$\text{Ext}(x) = \{y \in L_v \mid x \subseteq y\}.$$

For $X \subseteq L_v$, write $\text{Ext}(X) := \bigcup_{x \in X} \text{Ext}(x)$. We say x and y are *equivalent* if and only if $\text{Ext}(x) = \text{Ext}(y)$.

Interpretation. A completion is what you get when you add extra detail without contradicting what you already said. The extension $\text{Ext}(x)$ is the menu of all such future refinements.

Definition S1.4 (Weakness). Fix a vocabulary $v \subseteq \mathcal{P}$ and a statement $l \in L_v$. The *weakness* of l is the cardinality of its extension

$$w(l) := |\text{Ext}(l)|.$$

When $w(l_1) > w(l_2)$ we say l_1 is *weaker* than l_2 .

Remark S1.5 (Weakness does not imply inclusion). Weakness compares only extension sizes. In general $w(l_1) \geq w(l_2)$ does not imply $l_1 \subseteq l_2$. It also does not imply $\text{Ext}(l_1) \supseteq \text{Ext}(l_2)$. However, if $l_1 \subseteq l_2$ then $\text{Ext}(l_2) \subseteq \text{Ext}(l_1)$ and hence $w(l_1) \geq w(l_2)$.

Interpretation. Weakness counts how many further commitments you can still make while staying correct inside the same language. It is degrees of freedom that remain open. It is the invariant that survives re-encoding.

Definition S1.6 (Encoding map). Fix a vocabulary $v \subseteq \mathcal{P}$. Define the *encoding map* $\text{Enc}_v : \Phi \rightarrow L_v$ by

$$\text{Enc}_v(\phi) := \{p \in v \mid \phi \in p\}.$$

Interpretation. $\text{Enc}_v(\phi)$ lists every basic constraint in the vocabulary that is true in the current situation. It is the most detailed description the vocabulary can express.

Lemma S1.7 (Encoding is the maximal true statement). *For every $\phi \in \Phi$, the set $\text{Enc}_v(\phi)$ is true at ϕ . If $l \in L_v$ is also true at ϕ , then $l \subseteq \text{Enc}_v(\phi)$. Equivalently, for any $l \in L_v$,*

$$\phi \in T(l) \iff l \subseteq \text{Enc}_v(\phi).$$

Interpretation. Truth in the language is completely determined by the maximal vocabulary description.

Definition S1.8 (Realised encoding set). Fix a finite vocabulary v on environment Φ . Define the realised encoding set as the image of the encoding map

$$\text{Im}(\text{Enc}_v) := \{\text{Enc}_v(\phi) \mid \phi \in \Phi\} \subseteq L_v.$$

Definition S1.9 (Encoding capacity). For a finite vocabulary v , define its *encoding capacity* in bits by

$$I(v) := \log_2 |\text{Im}(\text{Enc}_v)|.$$

Interpretation. $I(v)$ is the log count of distinct vocabulary encodings that actually occur in the environment. It is a hard ceiling on how many internal distinctions that vocabulary can represent.

Definition S1.10 (Patch capacity ceiling). A bounded physical region $\mathcal{R}$ admits at most $N_{\max}(\mathcal{R})$ reliably distinguishable internal states. Define its bit ceiling as

$$I_{\max}(\mathcal{R}) := \log_2 N_{\max}(\mathcal{R}).$$

A vocabulary is realisable inside $\mathcal{R}$ only if $I(v) \leq I_{\max}(\mathcal{R})$.

Interpretation. If the patch cannot support enough distinguishable internal states, then the joint description you want is not physically instantiable there. More computation does not create more distinguishable memory states.

Definition S1.11 (Join vocabulary). For vocabularies v_1 and v_2 on the same environment, define the join

$$v_1 \vee v_2 := v_1 \cup v_2.$$

Lemma S1.12 (Join capacity is subadditive). For finite vocabularies v_1 and v_2 ,

$$I(v_1 \vee v_2) \leq I(v_1) + I(v_2).$$

Proof. For $c \in \text{Im}(\text{Enc}_{v_1 \vee v_2})$ define

$$r(c) := (c \cap v_1, c \cap v_2) \in \text{Im}(\text{Enc}_{v_1}) \times \text{Im}(\text{Enc}_{v_2}).$$

The map r is injective. So $|\text{Im}(\text{Enc}_{v_1 \vee v_2})| \leq |\text{Im}(\text{Enc}_{v_1})| |\text{Im}(\text{Enc}_{v_2})|$ and taking $\log_2$ gives the result. $\square$

Interpretation. The join cannot have more distinct encodings than the number of pairs of encodings its parts can realise. So joint descriptions can overflow a finite patch even when each partial description fits on its own.

Remark S1.13. In general L_v is a strict subset of 2^v . There can be bundles of programs that are mutually inconsistent, meaning their truth set is empty. That is why statements are defined by the nonempty truth condition.

S2 Capacity complementarity from joins

This section proves a representational form of complementarity. The result only uses finite encodings and a capacity ceiling. It does not assume quantum theory.

S2.1 Join capacity bounds

Join vocabularies and the basic capacity inequality are part of the shared kernel. See Definition S1.11 and Lemma S1.12.

Definition S2.1 (Encoding mutual information). Define

$$\text{MI}(v_1; v_2) := I(v_1) + I(v_2) - I(v_1 \vee v_2) = \log_2 \frac{|\text{Im}(\text{Enc}_{v_1})| |\text{Im}(\text{Enc}_{v_2})|}{|\text{Im}(\text{Enc}_{v_1 \vee v_2})|}.$$

Remark S2.2. This MI depends only on which encoding pairs are physically compatible. It does not depend on a probability distribution over Φ .

Theorem S2.3 (Capacity complementarity). *Fix a region $\mathcal{R}$ with ceiling $l_{\max}(\mathcal{R})$. Let v_1 and v_2 be vocabularies realisable in $\mathcal{R}$. If*

$$l(v_1) + l(v_2) - \text{MI}(v_1; v_2) > l_{\max}(\mathcal{R}),$$

then the join vocabulary $v_1 \vee v_2$ is not realisable in $\mathcal{R}$.

Proof. By definition of MI, the left side is $l(v_1 \vee v_2)$. If the join were realisable, I would have $l(v_1 \vee v_2) \leq l_{\max}(\mathcal{R})$. This contradicts the inequality. $\square$

Theorem S2.3 is the core representational claim. If two descriptions each sit near the same ceiling and share little redundancy, their join is an unphysical request.

S3 Window semantics and single patch realisation

This section now matches the main paper’s theorem-facing distinction. The issue is not merely whether the AMPS ingredients appear somewhere in the observer history. It is whether the late mode, the interior partner, the decoded purifier, and the decoder selector are jointly present in one patch state. I therefore move from generic ingredient sets to the explicit verification family and then add the bounded-history version needed for the time-sharing objection. The second step is where the recent *Time Is Space* identities matter most [1, 2, 3].

Definition S3.1 (Single patch verifier). A *single patch verifier* is a protocol whose decoding choice and verification outcome are both implemented by degrees of freedom confined to one infaller causal patch at the moment of verification. External preprocessing is allowed only insofar as its output is carried into the patch as physical state.

Remark S3.2. If you allow a distributed protocol, then you have already granted complementarity. You have accepted that no single patch contains the joint refinement of the exterior and interior descriptions.

S3.1 Objective trajectories and windows

I use the same window semantics as in the Technical Appendices and in *A Mind Cannot Be Smeared Across Time* [3, 1].

Definition S3.3 (Objective time and objective trajectories). Objective time is indexed by $u \in \mathbb{N}$. An *objective trajectory* is a function $\tau: \mathbb{N} \rightarrow \Phi$ such that $\tau(u+1) \neq \tau(u)$ for all u .

Remark S3.4 (Time as change at a layer). Objective time u counts physical transitions of the environment. The formalism also defines a layer-dependent time for each vocabulary. Given an objective trajectory τ and a vocabulary v , the induced layer trajectory is the encoding sequence $u \mapsto \text{Enc}_v(\tau(u))$. The layer clock ticks only when this encoding changes [3]. In plain English, if nothing you can represent changes, then no time passes at that layer.

Remark S3.5 (Why bounded serial time is not a free resource). The formalism already lets you compile bounded histories into derived state descriptions. The recent *Time Is Space* result proves two exact identities. Cumulative layer space over a bounded window is ordinary layer space in a lifted vocabulary. Elapsed layer time over that same window is layer space in a tick vocabulary [2]. In plain language, a bounded time budget is just another way of counting distinctions. That is the formal backdrop for the history-key result below.

Remark S3.6 (Notation). I write $\mathbb{N}_{>0} := \mathbb{N} \setminus \{0\}$ for the positive integers.

Definition S3.7 (Windowing). Fix a horizon $\Delta \in \mathbb{N}_{>0}$ and a stride $s \in \mathbb{N}_{>0}$. Define the window trajectory $\sigma^{\Delta,s}: \mathbb{N} \rightarrow \Phi^{\Delta+1}$ by

$$\sigma^{\Delta,s}(t) := (\tau(st), \tau(st+1), \dots, \tau(st+\Delta)).$$

I refer to $\sigma := \sigma^{\Delta,s}(t)$ as a window.

S3.2 Verification ingredient family

Definition S3.8 (Verification ingredient family). Fix a patch-level environment Φ_{patch} . Let $p_A, p_B, p_P, p_K \subseteq \Phi_{\text{patch}}$ be programs with the following meanings. p_A means the interior partner A is physically available in the patch. p_B means the late mode B is physically available in the patch. p_P means the decoded purifier of B is physically available in the patch. p_K means the decoder selector is physically instantiated in a control state that fixes the decoding channel. Write

$$X_{\text{ver}} := \{p_A, p_B, p_P, p_K\}.$$

Interpretation. The AMPS contradiction is not triggered by seeing these ingredients one at a time. It needs a single patch state that carries all of them together. X_{ver} is the minimum ingredient family needed to state that requirement cleanly.

S3.3 Occurrence, co-instantiation, and one-patch verification

Definition S3.9 (Occurrence and co-instantiation in a verification window). Fix a bounded objective window $\sigma = (\phi_0, \phi_1, \dots, \phi_\Delta) \in \Phi_{\text{patch}}^{\Delta+1}$. Define

$$\text{Occur}(X_{\text{ver}}, \sigma) \iff \forall p \in X_{\text{ver}} \exists k \leq \Delta \text{ such that } \phi_k \in p,$$

and

$$\text{CoInst}(X_{\text{ver}}, \sigma) \iff \exists k \leq \Delta \text{ such that } \phi_k \in \bigcap_{p \in X_{\text{ver}}} p.$$

Proposition S3.10 (One-observer verification needs co-instantiation). *If a protocol yields $\text{Occur}(X_{\text{ver}}, \sigma)$ but not $\text{CoInst}(X_{\text{ver}}, \sigma)$, then it does not produce a single-patch AMPS contradiction. It produces only a distributed history in which the ingredients appear across the window without one joint physical record.*

Proof. A single-patch verifier, by Definition S3.1, must carry both the decoding choice and the verification outcome inside one patch at the moment of verification. If $\text{CoInst}(X_{\text{ver}}, \sigma)$ fails, then there is no objective time-slice in the patch where A , B , the decoded purifier, and the decoder selector are jointly instantiated. So there is no patch state that contains the whole comparison record. The protocol therefore establishes at most ingredient-wise occurrence across the window. That is weaker than a single-observer contradiction. $\square$

Interpretation. This is the exact formal version of the one-observer versus distributed-protocol split. An arpeggio is not enough. The AMPS contradiction needs a chord.

Remark S3.11 (Temporal-gap normal form). The proposition above is the same structural separation formalised in *A Mind Cannot Be Smeared Across Time*. Within-window occurrence is strictly weaker than co-instantiation of the grounded conjunction [1]. For the firewall protocol, a chord is required. An arpeggio is not enough.

S3.4 Bounded serialisation and history vocabularies

One natural objection is that the observer could write the key gradually, or cycle through candidate decoders over time, and so avoid holding the whole selector at once. The right response is to count protocol-relevant histories rather than static labels.

Definition S3.12 (Protocol-relevant control histories). Fix a bounded preverification window of $\Delta + 1$ internal update steps. For each decoder $D \in \mathcal{D}$, let

$$h_D = (\sigma_0^{(D)}, \sigma_1^{(D)}, \dots, \sigma_\Delta^{(D)})$$

be the protocol-relevant control history of the verifier over that window. Identify two such histories when no protocol-relevant measurement at tolerance ε can distinguish them. Let $\mathcal{H}_\varepsilon$ be the set of resulting history classes.

Proposition S3.13 (Serial control histories still cost key bits). *If distinct decoders in $\mathcal{D}$ remain operationally distinct for the verification task, then $|\mathcal{H}_\varepsilon| \geq |\mathcal{D}|$. Consequently any vocabulary $v_{\mathcal{H}}$ that distinguishes these history classes satisfies*

$$I(v_{\mathcal{H}}) \geq \log_2 |\mathcal{D}|.$$

In particular, serialising decoder choice over a bounded preverification window cannot reduce the minimum distinction budget below the static key bound.

Proof. Map each decoder $D \in \mathcal{D}$ to its history class $[h_D] \in \mathcal{H}_\varepsilon$. If two operationally distinct decoders mapped to the same history class, then the protocol would provide no protocol-relevant way to distinguish them over the whole window at tolerance ε . That contradicts the assumption that they are operationally distinct for the verification task. So the map is injective and $|\mathcal{H}_\varepsilon| \geq |\mathcal{D}|$. Any vocabulary that distinguishes these classes therefore has at least $|\mathcal{D}|$ realised encodings, so by Definition S1.9 its encoding capacity is at least $\log_2 |\mathcal{D}|$. $\square$

Interpretation. If time multiplexing really distinguishes among $|\mathcal{D}|$ decoder choices, then it realises at least $|\mathcal{D}|$ protocol-relevant histories. The distinction budget has moved from a static register to a history vocabulary. It has not disappeared.

Remark S3.14 (Why *Time Is Space* matters here). Proposition S3.13 is the bookkeeping version of the claim that serial time is not free memory. Over any bounded window, cumulative layer space is ordinary layer space in a lifted vocabulary and elapsed layer time is layer space in a tick vocabulary [2]. So the history vocabulary above is not an ad hoc trick. It is the canonical Stack-Theoretic way of recompiling bounded serial distinction.

S4 Additional AMPS derivations and robustness

Section S3 handles the structural one-observer issue. This section keeps the rest of the ledger focused on coverage, approximation, and the toy bookkeeping models.

S4.1 Success fraction

The main text states the cleanest version in which the protocol succeeds for every microstate in a fixed orthonormal basis of the compatible microstate Hilbert space. If I allow a failure fraction $\eta \in (0, 1)$, interpret this as requiring success on at least $(1 - \eta) 2^{S_{\text{BH}}}$ basis microstates. This does not by itself guarantee success on arbitrary superpositions of the successful basis states. Uniform code subspace success is strictly stronger, and it is the notion controlled by alpha-bit reconstruction bounds. For the counting argument I only need a bound on how many basis microstates one decoder can cover. If each decoder succeeds uniformly only on code subspaces of dimension at most $d_\varepsilon^{\text{max}}$, then each decoder can cover at most $d_\varepsilon^{\text{max}}$ basis microstates.

Under this assumption, the same dimension counting gives

$$|\mathcal{D}| \geq \frac{(1 - \eta) 2^{S_{\text{BH}}}}{d_\varepsilon^{\text{max}}}.$$

In the alpha-bit regime, $\log_2 d_\varepsilon^{\text{max}} \leq \alpha_* S_{\text{BH}} + c_\varepsilon$ (the assumptions box in the main text), so $d_\varepsilon^{\text{max}} \leq 2^{\alpha_* S_{\text{BH}} + c_\varepsilon}$ [4, 5, 6]. Therefore

$$|\mathcal{D}| \geq (1 - \eta) \cdot 2^{(1 - \alpha_*) S_{\text{BH}} - c_\varepsilon}.$$

This changes the key bound only by an additive $O(1)$ term for fixed $\eta < 1$ because $\log_2(1 - \eta)$ and c_ε do not scale with S_{BH} .

S4.2 Approximate programming

Exact programming is not physically required. The relevant requirement is that the verifier can select among channels that remain operationally distinct at the target accuracy. Quantitative no-programming results lower bound the required program dimension as a function of the error parameter [7, 8]. For the black hole scaling, any polynomial dependence on $1/\varepsilon$ is negligible compared to the exponential dependence on S_{BH} .

S4.3 Why the alpha-bit bound is the right mainstream anchor

A random unitary scrambling model gives one way to argue that a single state-independent interior decoder cannot work on a code subspace that is too large. The alpha-bit regime is stronger as a mainstream anchor. It is a controlled statement inside AdS/CFT and it is natively phrased in the right language, namely code subspaces and state dependence.

The main text uses alpha-bits as an explicit upper bound on the code subspace dimension $d_\varepsilon^{\text{max}}$ for a fixed decoder. If future work produces tighter bounds for interior reconstruction, the key and patch size bounds strengthen automatically.

S5 Toy computations and figure generation

This section documents the computations used to generate Figure 1 and Extended Data Figures 1 and 2. Extended Data Figures 1 and 2 are not simulations of quantum gravity, AdS/CFT, or black hole evaporation. They are finite toy models that make the information bookkeeping in the main theorem concrete.

S5.1 Computing encoding capacity in a finite model

I pick a finite environment Φ and a finite vocabulary v . For each state $\phi \in \Phi$ I compute the induced encoding $\text{Enc}_v(\phi)$, which is the set of programs in v that contain ϕ . I then count how many distinct encodings occur. This count is $|\text{Im}(\text{Enc}_v)|$. The encoding capacity is $\mathsf{l}(v) = \log_2 |\text{Im}(\text{Enc}_v)|$.

In the implementation, each program is represented extensionally as a bitset over Φ .

Interpretation. A bitset is a compact way to represent a subset of a finite set. It is a binary mask with one bit per element of Φ . A one means the element is in the program. A zero means it is not. Each $\text{Enc}_v(\phi)$ is represented as a bit mask over the program indices. The script enumerates all $\phi \in \Phi$ and collects the distinct masks.

S5.2 Extended Data Fig. 1

Extended Data Fig. 1 is a toy demonstration of capacity complementarity. I pick a finite environment Φ that is a Boolean cube of dimension $\text{env_n} = 12$, so it has $|\Phi| = 2^{12}$ mutually exclusive states. In math notation, $\Phi = \{0, 1\}^{\text{env_n}}$. **Interpretation.** This environment is just the set of all 12 bit strings. There are 2^{12} such strings. I then sample two random vocabularies v_1 and v_2 . Each vocabulary contains $m = 6$ programs. Each program is generated by including each environment state independently with probability $p = 0.2$. For each sampled pair, I form the join vocabulary $v_1 \vee v_2$ and compute its encoding capacity $\mathsf{l}(v_1 \vee v_2)$ in bits.

I draw a fixed illustrative ceiling $\mathsf{l}_{\text{max}} = 9.8$ bits. Each vocabulary individually has $\mathsf{l} \leq 6$ bits by construction because it contains only 6 programs. The join $v_1 \vee v_2$ has up to 12 programs and can therefore exceed the same ceiling. I repeat this for 200 samples with random seed 2.

S5.3 Extended Data Fig. 2

Extended Data Fig. 2 illustrates the decoder counting step used in the main text. I model the microstate basis as a finite set H with $|H| = 2^n$. You can think of n as the number of microstate bits. I assume one decoder can succeed on at most $d_{\text{max}} = 2^{\alpha n}$ microstates. In Supplementary Code 1 I set $d_{\text{max}} := \lfloor 2^{\alpha n} \rfloor$ so the subset size is an integer and does not exceed $2^{\alpha n}$.

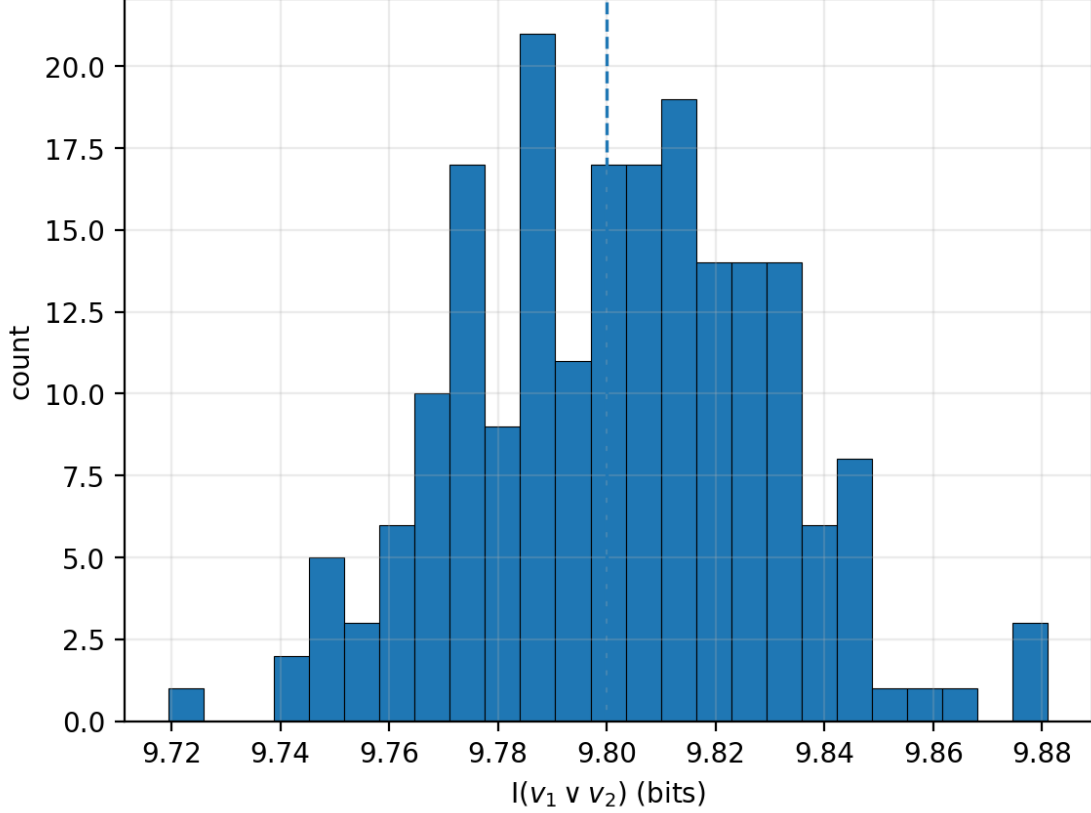


Figure 1: **Extended Data Fig. 1 — Toy complementarity from a finite capacity ceiling.** The horizontal axis is the join capacity $I(v_1 \vee v_2)$ in bits. The vertical axis is the number of random samples in the histogram bin. The vertical line is the ceiling $I_{\max} = 9.8$ bits. Parameters are `env_n = 12`, $m = 6$, $p = 0.2$, `samples = 200`, `seed = 2`. Source data are provided in `source_data/source_data_ED_fig1_toy_complementarity.csv`. Each vocabulary has 6 programs so it fits under the ceiling. The join has up to 12 programs and can exceed the same ceiling. This is the finite form of capacity complementarity proved in Section S2.

A covering argument then gives a lower bound. Covering all microstates requires at least $\lceil |H|/d_{\max} \rceil$ decoders, even in the best case with no overlap. So the key size obeys $\log_2 |\mathcal{D}| \geq \log_2 \lceil |H|/d_{\max} \rceil$. In the idealised scaling where $d_{\max} = 2^{\alpha n}$ exactly, this reduces to $\log_2 |\mathcal{D}| \geq (1 - \alpha)n$.

The figure also plots a random overlap model. In that model, each decoder is a random subset of size $d_{\max}$. I sample many such subsets and then run a greedy set cover algorithm to cover H . Set cover means choosing a collection of subsets whose union is all of H . Greedy means I repeatedly pick the decoder that covers the largest number of microstates that are still uncovered. Because random subsets overlap, the greedy cover typically requires more decoders than the lower bound.

The parameter values in Supplementary Code 1 are $\alpha = 0.6$, $n \in \{6, 8, 10, 12\}$, factor 12, trials 2, and random seed 2.

Interpretation. The parameter n sets the microstate set size $|H| = 2^n$. The parameter α sets the per-decoder coverage size $d_{\max} = \lceil 2^{\alpha n} \rceil$. **Warning about notation.** This toy exponent parameter α is not the same object as the theorem parameter α_* . The theorem parameter α_* comes from competing quantum extremal surfaces. The factor parameter sets how many random candidate decoders I generate before running greedy cover. The script uses $M = \text{factor} \times \lceil |H|/d_{\max} \rceil$ random subsets. Trials is the number of independent greedy covers I run and average. The seed fixes the randomness so the plot is reproducible.

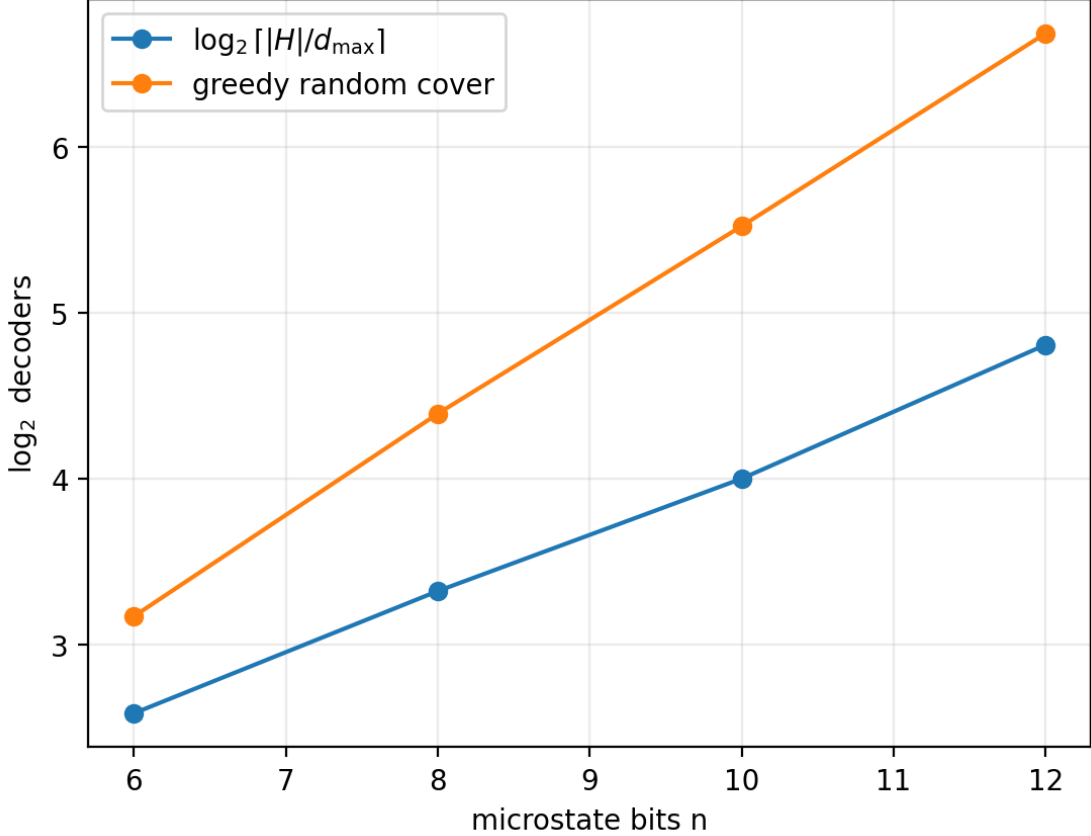


Figure 2: **Extended Data Fig. 2 — Decoder coverage implies key bits.** The horizontal axis is n where $|H| = 2^n$. The vertical axis is $\log_2$ of the number of decoders. The lower bound $\log_2 |\mathcal{D}| \geq \log_2 [|H|/d_{\max}]$ scales linearly in n . The random set cover toy model typically requires more decoders than this bound because of overlap. Parameters are $\alpha = 0.6$, $n \in \{6, 8, 10, 12\}$, factor = 12, trials = 2, seed = 2. Source data are provided in `source_data/source_data_ED_fig2.decoder_cover_scaling.csv`.

S5.4 Figure 1 in the main text

Figure 1 is a plot of the asymptotic large entropy form of the main theorem in the main text. In the simplified regime where $d_{\epsilon}^{\max} \approx 2^{\alpha_* S_{\text{BH}}}$ and additive constants are negligible, the theorem reduces to $\rho^2 \gtrsim 1 - \alpha_*$. Equivalently $\rho_{\min} = \sqrt{1 - \alpha_*}$.

Supplementary Code 1 samples 400 evenly spaced values of α_* between 0 and 1 and plots the resulting curve.

S5.5 Reproducibility

The computations are implemented in Supplementary Code 1, a short Python script that regenerates the toy figures and CSV source-data files. That code archive is intentionally not included in this source bundle because the author will attach it separately at submission. The present bundle therefore contains the manuscript source, figure assets, and source-data CSVs only. The script depends on `numpy` and `matplotlib`. The output figures and the CSV source-data files are deterministic given the script random seeds. The CSV files are written under `source_data` and correspond to Figure 1 in the main text and Extended Data Figures 1 and 2.

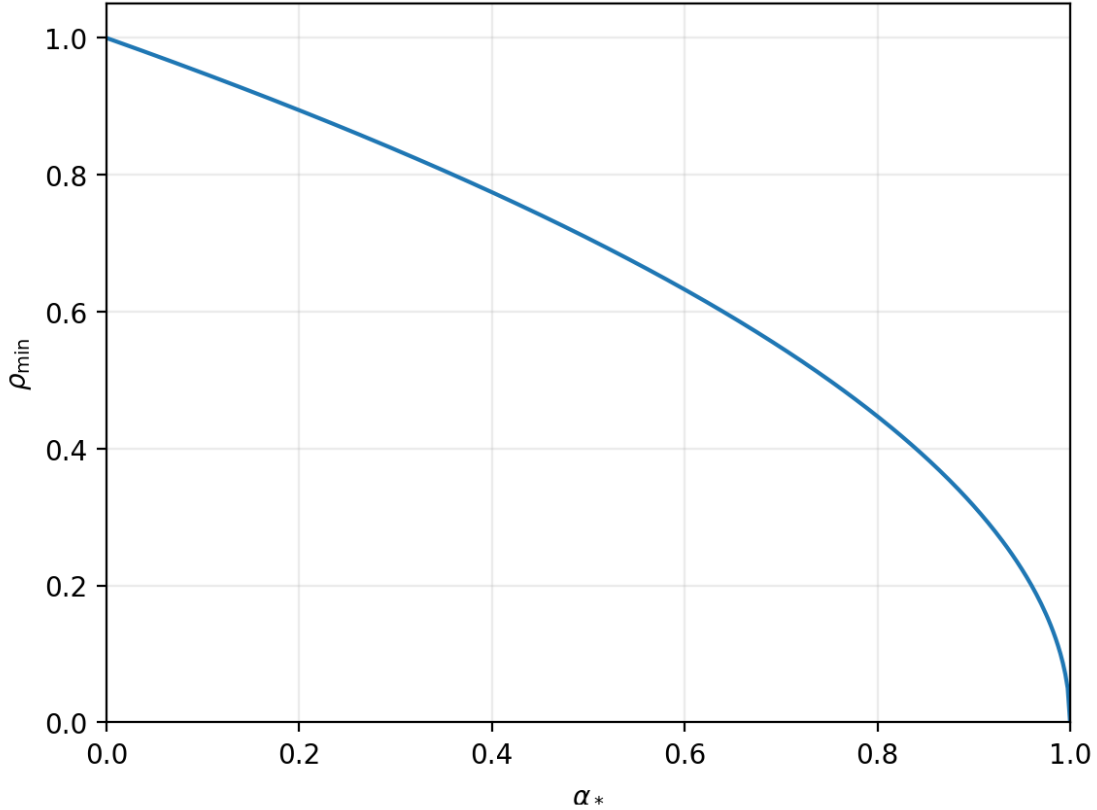


Figure 3: **Figure 1 (main text) — Asymptotic patch size bound.** The horizontal axis is α_* . The vertical axis is $\rho_{\min} = \sqrt{1 - \alpha_*}$. The curve is evaluated on 400 evenly spaced values of α_* . Source data are provided in `source_data/source_data_Fig1_rho_min_vs_alpha.csv`. When α_* is bounded away from one, any successful verifier must control a horizon-scale patch.

S6 Outlook on capacity constraints in quantum gravity

This paper most certainly does *not* claim to solve quantum gravity. It merely isolates one operational point. If a paradox requires a single observer to instantiate more control information than can fit inside their causal patch, then the paradox is a request for an unphysical experiment.

The bookkeeping formalism used here is one way to express the conclusion, and it is useful because it makes it relatively easy to separate, classify, and define the relevant ingredients. The first ingredient is a description, meaning a set of distinctions that could be implemented by some physical degrees of freedom. The second ingredient is a capacity ceiling, meaning a covariant upper bound on how many such distinctions can coexist inside one bounded region. The third ingredient is redundancy, meaning how much two descriptions overlap so that their join costs less than the sum of their individual costs.

In quantum gravity language, the covariant entropy bound supplies the ceiling [9]. Entanglement wedge reconstruction and alpha-bit bounds supply a concrete handle on redundancy in controlled settings [4]. The main theorem is one example of what drops out when you combine a ceiling with a redundancy bound.

One possible route from this point toward a more complete theory is to treat covariant capacity constraints as kinematic consistency conditions. On this view, geometry is not only a stage on which information lives. Geometry is also a ledger of which joint descriptions can be simultaneously realised.

Three concrete next steps look both feasible and falsifiable.

1. Derive island-style reconstruction thresholds as explicit join constraints. In AdS/CFT plus bath models,

one can ask which combinations of interior and exterior statements can be jointly realised in one patch at fixed error. This reframes the usual story as an inequality over capacities rather than a narrative about observers.

2. Replace the universal success requirement by a tunable success fraction and match it to known code subspace constructions. This would turn the key bound into a family of tradeoff curves. Those curves can be compared directly to explicit state-dependent reconstruction schemes.
3. Extend the control capacity ledger beyond black holes. The same separation between computation and control should apply to any protocol that relies on choosing among exponentially many effective channels. If the framework is right, similar ceilings should appear in cosmological horizons and in any scenario where the covariant entropy bound is tight.

Again I am not claiming this bookkeeping formalism is the only way to write the argument. The substantive claim is simpler. Whatever quantum gravity is, it should respect covariant information accounting. This formalism is one way to do that accounting without smuggling in hidden assumptions about what an observer can store.

References

- [1] Michael Timothy Bennett. “A Mind Cannot Be Smeared Across Time”. In: *Forthcoming in Proceedings of the AAAI 2026 Spring Symposium on Machine Consciousness: Integrating Theory, Technology, and Philosophy*. Burlingame, California, USA: AAAI Press, Apr. 2026.
- [2] Michael Timothy Bennett. *Time Is Space*. 2026.
- [3] Michael Timothy Bennett. *Technical Appendices*. Version 6.21. Archived release on Zenodo. Source repository: <https://github.com/ViscousLemming/Technical-Appendices>. 2025. DOI: 10.5281/zenodo.7641741. URL: <https://doi.org/10.5281/zenodo.7641741> (visited on 03/23/2026).
- [4] Patrick Hayden and Geoffrey Penington. “Learning the Alpha-bits of Black Holes”. In: *Journal of High Energy Physics* 2019.12 (2019), p. 007. DOI: 10.1007/jhep12(2019)007. arXiv: 1807.06041 [hep-th]. URL: [https://doi.org/10.1007/jhep12\(2019\)007](https://doi.org/10.1007/jhep12(2019)007).
- [5] Geoffrey Penington. “Entanglement wedge reconstruction and the information paradox”. In: *Journal of High Energy Physics* 2020.9 (2020), p. 002. DOI: 10.1007/jhep09(2020)002. arXiv: 1905.08255 [hep-th]. URL: [https://doi.org/10.1007/jhep09\(2020\)002](https://doi.org/10.1007/jhep09(2020)002).
- [6] Ahmed Almheiri et al. “The entropy of Hawking radiation”. In: *Reviews of Modern Physics* 93.3 (2021), p. 035002. DOI: 10.1103/revmodphys.93.035002. arXiv: 2006.06872 [hep-th]. URL: <https://doi.org/10.1103/revmodphys.93.035002>.
- [7] Michael A. Nielsen and Isaac L. Chuang. “Programmable quantum gate arrays”. In: *Physical Review Letters* 79.2 (1997), pp. 321–324. DOI: 10.1103/physrevlett.79.321. arXiv: quant-ph/9703032. URL: <https://doi.org/10.1103/physrevlett.79.321>.
- [8] Aleksander M. Kubicki, Carlos Palazuelos, and David Pérez-García. “Resource Quantification for the No-Programming Theorem”. In: *Physical Review Letters* 122 (2019), p. 080505. DOI: 10.1103/physrevlett.122.080505. arXiv: 1805.00756 [quant-ph]. URL: <https://doi.org/10.1103/physrevlett.122.080505>.
- [9] Raphael Bousso. “The holographic principle”. In: *Reviews of Modern Physics* 74.3 (2002), pp. 825–874. DOI: 10.1103/revmodphys.74.825. arXiv: hep-th/0203101. URL: <https://doi.org/10.1103/revmodphys.74.825>.